Relatório — Laboratório 2: limites do processo (heap e stack)

Disciplina: Sistemas Operacionais · TADS/IFRN · 2026.1
Aluno(a): _____
Data: //_____

> Endereços e medidas variam conforme arquitetura, compilador e sistema. As tabelas abaixo registram medições feitas neste workspace em Linux x86-64, com GCC e as flags pedidas no enunciado (`-Wall -Wextra -O0`). Os valores do PDF do laboratório são referências; não são apresentados como medições locais.

Previsões e resultados

As hipóteses foram registradas antes da discussão dos resultados; a coluna da direita foi completada após as execuções. Os limites de stack e heap foram aplicados em subshells para não alterar o shell de trabalho.

Parte	Previsão	Resultado observado
1 — mapa	Código/string literal e globais devem ficar nas regiões do executável; local na stack e bloco dinâmico no heap.	Confirmado pelos endereços e intervalos de `/proc/<PID>/maps` da tabela da Parte 1.
2 — heap sem escrita	400 alocações de 1 MiB aumentam `VmSize` em cerca de 400 MiB, mas deixam `VmRSS` baixo.	`VmSize=413896 kB` e `VmRSS=3348 kB`; relação `VmSize/VmRSS ≈ 123,6`.
3 — limite do heap	Sob `ulimit -v`, um pedido recusado deve fazer `malloc()` retornar `NULL` e permitir encerramento tratado.	Com 262144 KiB, a alocação falhou no MiB nº 253; saída 1, sem depender do OOM killer.
4 — stack	A recursão deve alcançar o limite da stack antes de esgotar a RAM e terminar por sinal.	Com 8192 KiB, o último nível impresso foi 7500 (7967 KiB medidos); sinal `SIGSEGV`, saída 139.
5 — stack dobrada	Dobrar o limite deve aproximadamente dobrar a profundidade, mas não eliminar o estouro.	Com 16384 KiB, o último nível impresso foi 15000 (15936 KiB); também terminou com `SIGSEGV`, saída 139.
6 — arquivo	Streaming deve processar arquivo maior que a RAM disponível sem guardar tudo; a estratégia inteira depende de uma alocação proporcional ao arquivo.	Para 100 MiB, ambas somaram `13368675856`; RSS final foi 1684 kB (incremental) contra 104088 kB (inteiro). Sob limite virtual de 96 MiB, a estratégia inteira falhou com saída 1.

Parte 1 — Regiões de memória

As globais inicializada e zerada pertencem às seções `.data` e `.bss`; neste processo, ambas caíram na mesma página RW do executável, embora continuem sendo seções ELF distintas. O código ficou em `text`, a string literal em uma página de leitura, o bloco pequeno de `malloc()` no `[heap]` e a variável local na `[stack]`.

Item	Endereço observado	Faixa correspondente em `/proc/<PID>/maps`	Região/permissão
`main` (código)	`0x5b20034912a7`	`5b2003491000-5b2003492000`	executável `r-xp` (`text`)
String literal	`0x5b2003492008`	`5b2003492000-5b2003493000`	somente leitura (`rodata`)
Global inicializada	`0x5b2003494010`	`5b2003494000-5b2003495000`	RW (`data`/`bss`)
Global zerada	`0x5b200349402c`	`5b2003494000-5b2003495000`	RW (`data`/`bss`)
`malloc(4096)`	`0x5b203edd92a0`	`5b203edd9000-5b203edfa000`	`[heap]`
Variável local	`0x7ffeaaf7b83c`	`7ffeaaf5c000-7ffeaaf7e000`	`[stack]`

As linhas de `maps` foram capturadas enquanto o processo estava parado; ASLR faz os endereços mudarem em outra execução. A página RW que contém as globais mede 4 KiB; o mapa mostra permissões e páginas mapeadas, não separa necessariamente cada seção ELF em uma faixa própria.

`VmSize` contabiliza o espaço de endereçamento virtual reservado pelo processo. `VmRSS` contabiliza páginas residentes na memória física naquele momento. `VmData` é a memória classificada pelo kernel como dados do processo e `VmStk` informa o tamanho da stack.

No mesmo instante da captura, `/proc/<PID>/status` mostrou `VmSize=2696 kB`, `VmRSS=1732 kB`, `VmData=224 kB` e `VmStk=136 kB`.

Partes 2 e 3 — Crescimento e limite do heap

Os dois experimentos pediram 400 blocos de 1 MiB. A pausa entre blocos foi desativada (`./heap 400 nao manter 0` e `./heap 400 sim manter 0`) para obter as medidas finais sem esperar 400 segundos; o modo padrão continua pausando para permitir observação com `watch`.

Modo	`VmSize` final	`VmRSS` final	`VmData` final	`VmSize / VmRSS`
Reservar sem escrever	413896 kB	3348 kB	411424 kB	123,6×
Tocar cada bloco com	413896 kB	412948 kB	411424 kB	1,002×

Modo	`VmSize` final	`VmRSS` final	`VmData` final	`VmSize / VmRSS`
`memset()`				

Sem escrita, a faixa de endereços aumentou, mas apenas poucas páginas ficaram residentes; o RSS ficou em cerca de 3,3 MiB. Ao escrever nos blocos, as páginas foram usadas e o RSS ficou quase igual ao tamanho virtual. As razões acima comparam os valores finais de `VmSize` e `VmRSS`; os resultados variam um pouco com o ambiente.

Para o limite, executei `( ulimit -v 262144; ./heap 2048 sim manter 0 )`. O processo chegou a 248468 kB de RSS no registro de 240 MiB e `malloc()` falhou ao pedir o MiB nº 253. A mensagem de erro foi impressa e o código de saída foi **1**. O limite de 256 MiB valeu somente para o subshell. Isso demonstra uma falha controlável do alocador, diferente de deixar o processo crescer até um possível OOM killer.

Partes 4 e 5 — Crescimento e limite da stack

Cada chamada de `explode()` cria um quadro de ativação com endereço de retorno, parâmetros e variáveis locais. A recursão cresceu em direção a endereços menores até atingir o limite. As duas execuções foram realizadas com intervalo de impressão de 500 níveis:

Limite configurado	Último nível impresso	Stack usada aproximada	Código de saída
`ulimit -s 8192` (8192 KiB)	7500	7967 KiB	139 (`SIGSEGV`)
`ulimit -s 16384` (16384 KiB)	15000	15936 KiB	139 (`SIGSEGV`)

Dobrar o limite praticamente dobrou a profundidade impressa; a profundidade real é um pouco maior que o último múltiplo mostrado. O mesmo sinal ocorreu nos dois casos: aumentar o limite apenas adiou o estouro, não corrigiu a recursão infinita. A saída 139 corresponde a `128 + 11`, sendo 11 o número de `SIGSEGV` neste Linux.

O aviso do compilador sobre recursão infinita não impede a compilação: é um diagnóstico, não uma violação que torne o programa inválido em C. A recursão é intencional neste experimento, e `-O0` ajuda a evitar que a otimização transforme a chamada em um laço.

Parte 6 — Processamento de arquivo

Criei em `/tmp` um arquivo de teste de 100 MiB e rodei ambos os modos. A soma é idêntica, mas o RSS é bem diferente:

Estratégia	Soma dos bytes	`VmRSS` antes	`VmRSS` depois
Incremental, buffer reutilizado de 4 KiB	13368675856	1592 kB	1684 kB

Estratégia	Soma dos bytes	`VmRSS` antes	`VmRSS` depois
Inteira, alocação do tamanho do arquivo	13368675856	1592 kB	104088 kB

A estratégia inteira usou cerca de **61,8×** o RSS da incremental ao final. Com `( ulimit -v 98304; ./arquivo inteiro /tmp/lab2-grande.bin )`, `malloc()` falhou ao pedir os 100 MiB e o programa saiu com código 1. O limite foi 96 MiB porque, nesta máquina, 128 MiB ainda permitiu a alocação; o resultado depende do tamanho do executável e do ambiente. Para um CSV de 40 GB numa máquina com 8 GB de RAM, usaria a estratégia incremental/streaming, por linhas ou blocos. Carregar tudo de uma vez exige memória proporcional ao arquivo e não é uma estratégia viável nesse cenário.

Comparação final: heap e stack

Característica	Heap	Stack
Finalidade	Dados com tamanho ou duração definidos durante a execução	Chamadas de função, parâmetros e variáveis locais automáticas
Crescimento	Gerenciado pelo alocador; pode usar `brk` e/ou `mmap`	Quadros são adicionados e removidos conforme chamadas e retornos
Quem solicita	O programa solicita ao alocador (`malloc()`); o alocador obtém memória do kernel	Convenção de chamada e código gerado usam a região de stack já provida pelo sistema
Exemplo	Estrutura dinâmica ou buffer cujo tamanho é conhecido em runtime	Variável local de uma função
Liberação	Normalmente explícita com `free()`	Automática ao retornar da função
Limite	Espaço virtual, limites do processo/sistema e disponibilidade do alocador	Limite de stack do processo, configurável com `ulimit -s`
Falha possível	`malloc()` retorna `NULL` quando não consegue atender ao pedido	Estouro pode gerar `SIGSEGV`
Manifestação	Falha explícita, que o programa pode testar e tratar	Falha ao acessar a página que excede a região válida; o processo recebe um sinal

Perguntas de investigação

1. **Por que `VmSize` chegou a 413 MB com `VmRSS` em 3 MB?** O processo reservou um grande intervalo

de endereços virtuais, mas não escreveu nos blocos. Assim, podia endereçá-los, embora quase nenhuma página correspondente precisasse estar residente na RAM.

2. **Por que os limites falham de formas diferentes?** `malloc()` é uma solicitação explícita: pode detectar a impossibilidade e retornar `NULL`. A stack é usada implicitamente por chamadas e variáveis locais; ao ultrapassar seu limite, o acesso à memória não pode ser atendido e o kernel sinaliza o processo com `SIGSEGV`.
3. **Por que o compilador não impede a recursão infinita?** O padrão da linguagem permite recursão e o compilador normalmente apenas emite um aviso. A execução pode depender de entrada/condição; não é obrigação do compilador rejeitar esse código.
4. **Por que `.data` e `.bss` são separadas mesmo com endereços vizinhos?** Elas têm semânticas diferentes: `.data` contém valores inicializados que precisam ser carregados; `.bss` representa dados inicialmente zerados. Separá-las permite ao executável e ao carregador tratar inicialização, tamanho e mapeamento corretamente. Proximidade no espaço de endereços não significa que pertençam à mesma seção.
5. **O que aconteceria se heap e stack crescessem na mesma direção?** Poderiam consumir o espaço disponível um do outro mais cedo, limitando o crescimento de uma região mesmo quando a outra ainda tivesse pouca utilização. Crescer em direções opostas permite compartilhar o espaço de endereçamento disponível com mais flexibilidade.
6. **Como processar um CSV de 40 GB em uma máquina com 8 GB de RAM?** Usaria leitura incremental/streaming, por exemplo, linha a linha ou em blocos. Carregar tudo de uma vez requer memória proporcional ao arquivo e não cabe, além de precisar considerar memória adicional para estruturas de processamento.

Atividade de síntese — mapa e pergunta central

Esquema do mapa observado, fora de escala e com os intervalos medidos. O heap fica em endereços menores que as bibliotecas e a stack; heap e stack crescem um em direção ao outro no espaço de endereçamento disponível.

```
endereços baixos                                                    endereços altos
0x5b200349...      0x5b203edd...       0x75563f...              0x7ffeaaf...
[text/rodata/data/bss] ... [heap] ... [bibliotecas e mmap] ... [stack]
                                                                    heap cresce →
← stack cresce
                                        [heap] 5b203edd9000–5b203edfa000: 0x21000 = 132
KiB
                                        [stack] 7ffeaaf5c000–7ffeaaf7e000: 0x22000 = 136
KiB
```

Os tamanhos foram calculados subtraindo os endereços hexadecimais de cada intervalo de `maps`: heap `0x21000 / 1024 = 132 KiB`; stack `0x22000 / 1024 = 136 KiB`. São os tamanhos das faixas mapeadas nessa captura, não os limites máximos configurados para o processo.

Até onde um processo consegue crescer?

1. O processo cresce enquanto houver espaço virtual e os limites de recursos permitirem.
2. No heap, uma alocação recusada pode ser detectada com `malloc() == NULL`.
3. Na stack, exceder o limite pode terminar o processo com `SIGSEGV`.

4. O algoritmo importa: streaming processa arquivos grandes com memória de trabalho quase constante.

Desafio — liberar cada bloco

Foi adicionado ao `heap.c` o modo `./heap 8 sim liberar 0`, que toca cada bloco e chama `free()` logo em seguida. A execução de teste neste workspace mostrou:

- Inicial: `VmSize=2696 kB`, `VmRSS=1744 kB`, `VmData=224 kB`.
- Final: `VmSize=3720 kB`, `VmRSS=2768 kB`, `VmData=1248 kB`.

Neste teste, o `VmSize` **não** retornou ao valor inicial. Após os primeiros blocos, as medidas ficaram aproximadamente estáveis; isso é compatível com o alocador mantendo memória em sua arena/cache para reutilização, em vez de devolver imediatamente todas as páginas ao kernel. O resultado depende da implementação do alocador e do ambiente, então não é uma regra universal.

Para testar essa hipótese, pode-se comparar as chamadas de memória do processo com `strace` (`brk`, `mmap`, `munmap`) e observar `/proc/<PID>/maps` durante o experimento. Também se pode testar `malloc_trim(0)` (extensão da glibc) após as liberações e comparar `VmSize`/`VmRSS` antes e depois; se a memória retida for devolvida, os valores devem diminuir. O `free()` torna o bloco disponível para reutilização pelo alocador, mas não garante por si só que o processo reduzirá imediatamente o espaço mapeado.

Questões finais

1. **Diferença entre heap e stack:** o heap sustenta alocações dinâmicas cuja vida útil é controlada pelo programa; a stack sustenta chamadas de função e dados locais, liberados automaticamente no retorno.
2. **Diferença entre `VmSize` e `VmRSS`:** `VmSize` é espaço virtual total contabilizado; `VmRSS` é a parcela atualmente residente em memória física.
3. **Relação entre `malloc()` e heap:** `malloc()` é uma interface do alocador para pedir memória dinâmica. O alocador pode obter essa memória do kernel por diferentes mecanismos; nem todo `malloc()` corresponde necessariamente à região `[heap]` mostrada em `maps`.
4. **Relação entre chamadas e stack:** uma chamada normalmente cria um quadro de ativação na stack; o retorno remove esse quadro.
5. **Por que recursão excessiva gera `SIGSEGV`?** Cada chamada consome mais stack. Ao ultrapassar a região permitida, a próxima referência não pode ser atendida e o kernel sinaliza o processo. Diferente de `malloc()`, chamadas comuns não retornam um valor `NULL` informando que o quadro não coube.
6. **Por que um arquivo de 10 GB não exige 10 GB de RAM?** Se for processado em blocos, só é necessário manter na memória o bloco atual e o estado do cálculo, não o arquivo inteiro.
7. **Por que o sistema operacional controla recursos dos processos?** Para limitar e contabilizar recursos, proteger a máquina e permitir que programas reajam a limites. No experimento do heap, `ulimit -v` força uma falha controlável de `malloc()`; no da stack, `ulimit -s` limita a recursão e demonstra que um processo não pode crescer indefinidamente.

Conferência dos critérios de entrega

Critério	Onde está atendido
1. Previsões das Partes 1–6 e resultados	Tabela “Previsões e resultados”
2. Endereço × faixa de `maps`	Tabela da Parte 1
3. `VmSize`/`VmRSS` dos dois modos e razão	Tabela das Partes 2 e 3
4. Falha de `malloc()`, `ulimit -v` e código	Partes 2 e 3
5. Dois limites da stack e profundidades	Tabela das Partes 4 e 5
6. Duas estratégias, mesma soma e RSS diferente	Tabela da Parte 6
7. Comparação heap × stack	Tabela “Comparação final”
8. Seis perguntas de investigação	Seção “Perguntas de investigação”
9. Sete questões finais	Seção “Questões finais”
10. Mapa, cálculos e resposta em até cinco linhas	Seção “Atividade de síntese”
11. Desafio (opcional)	Seção “Desafio — liberar cada bloco” e `heap.c`

Observação de compilação

Os quatro programas compilam com `gcc -Wall -Wextra -O0`. `pilha.c` emite o aviso `infinite recursion detected`, esperado para a recursão intencional do experimento.